WAZUH

Proyecto: DevSecOps

Profesor: Rodrigo Canales Ramírez

Requerimientos Stakeholder

- Obtener vulnerabilidades de Wazuh para tener visibilidad sobre la trazabilidad en el tiempo
- Poseer seguridad de autenticación

Requisitos funcionales

Integración con Wazuh

Múltiples conexiones, filtros por CVE, agente, severidad

Persistencia histórica

Guardar timestamp, usuario, detalles de extracción

Seguridad

JWT, gestión de usuarios, hashing

Visualización

SPA, dashboard

Requisitos no funcionales

Escalabilidad

Soporte para diferentes Wazuh

Dockerización

Ejecución mediante Docker

Rendimiento

Tiempo de respuesta menor a 2 segundos

Mantenibilidad

Código organizado en módulos

Calidad

Pruebas unitarias

Componentes

API Backend

Scanner de seguridad: Análisis dinámico (DAST)

Fuente de vulnerabilidades

Base de datos

Scanner de seguridad: Análisis de código estático (SAST)

Entorno de contenedores

Automatización

Funcionalidades

wazuh.

Conexión con Wazuh

PostgreSQL

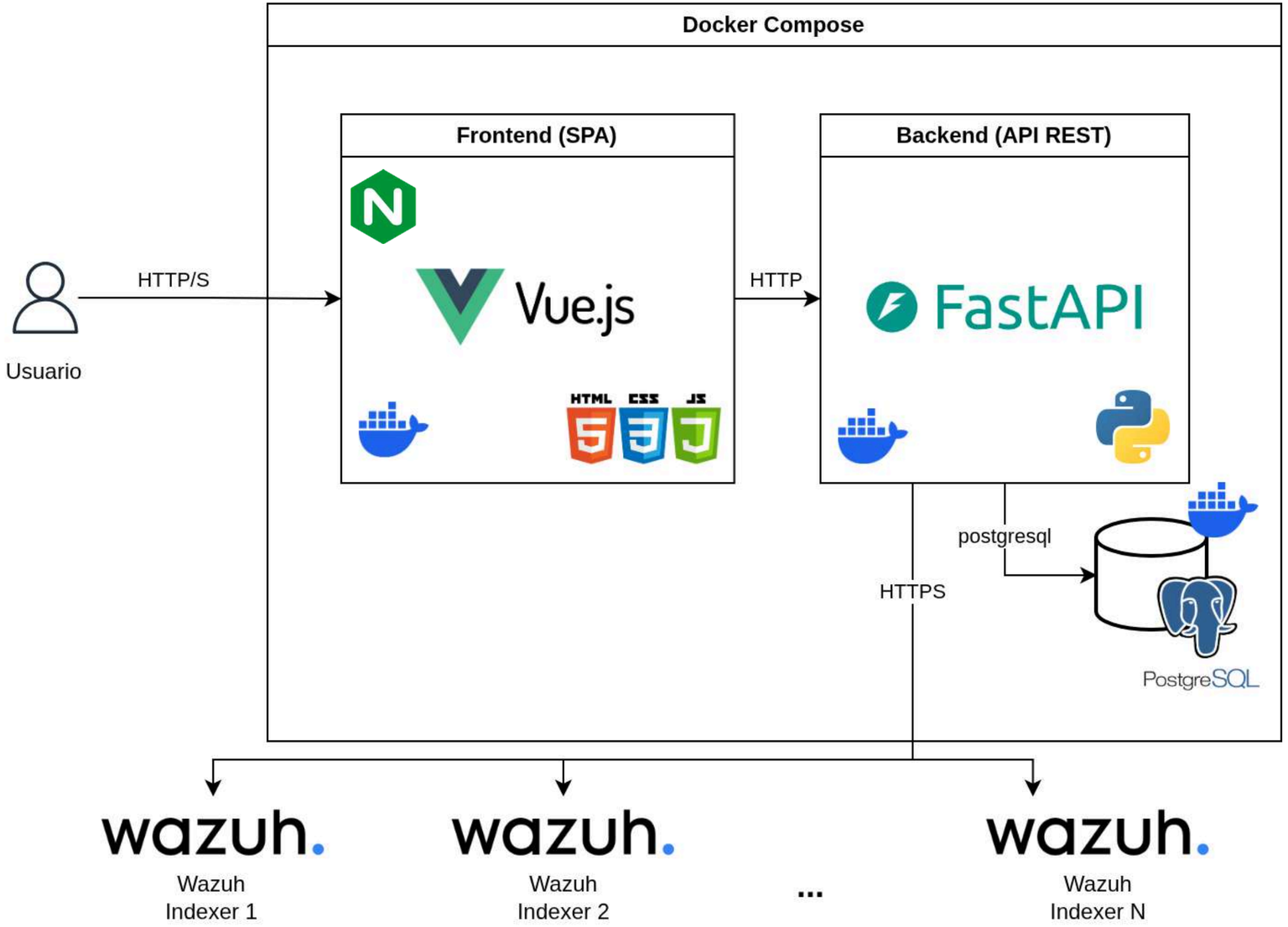
Persistencia de vulnerabilidades

JWT

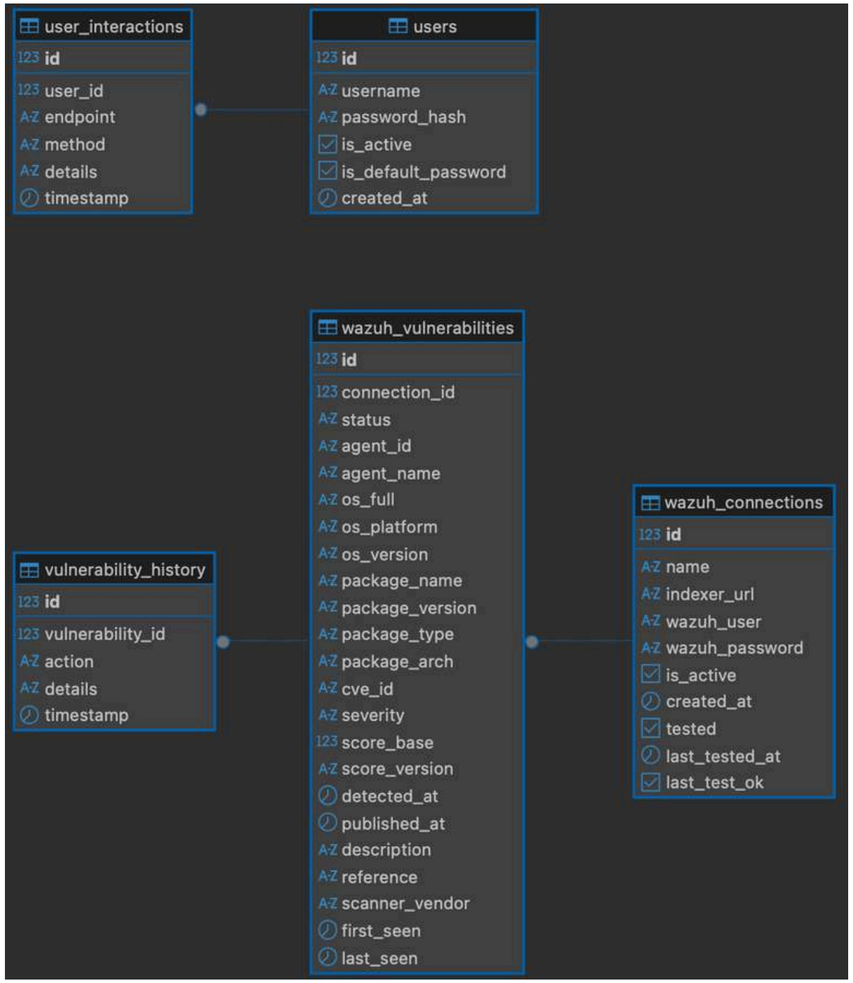
Control de accceso

Visualización de vulnerabilidades

Diagrama de arquitectura



Base de datos



Herramientas de construcción

Lenguajes

Python 3.12

Node 24

Gestor de dependencias

pip

npm

Frameworks

FastAPI + Uvicorn

Vue + Vite

Testing

Pytest

Vitest

Pipeline

Test & Cobertura

SAST

Quality Gate

Deploy

DAST

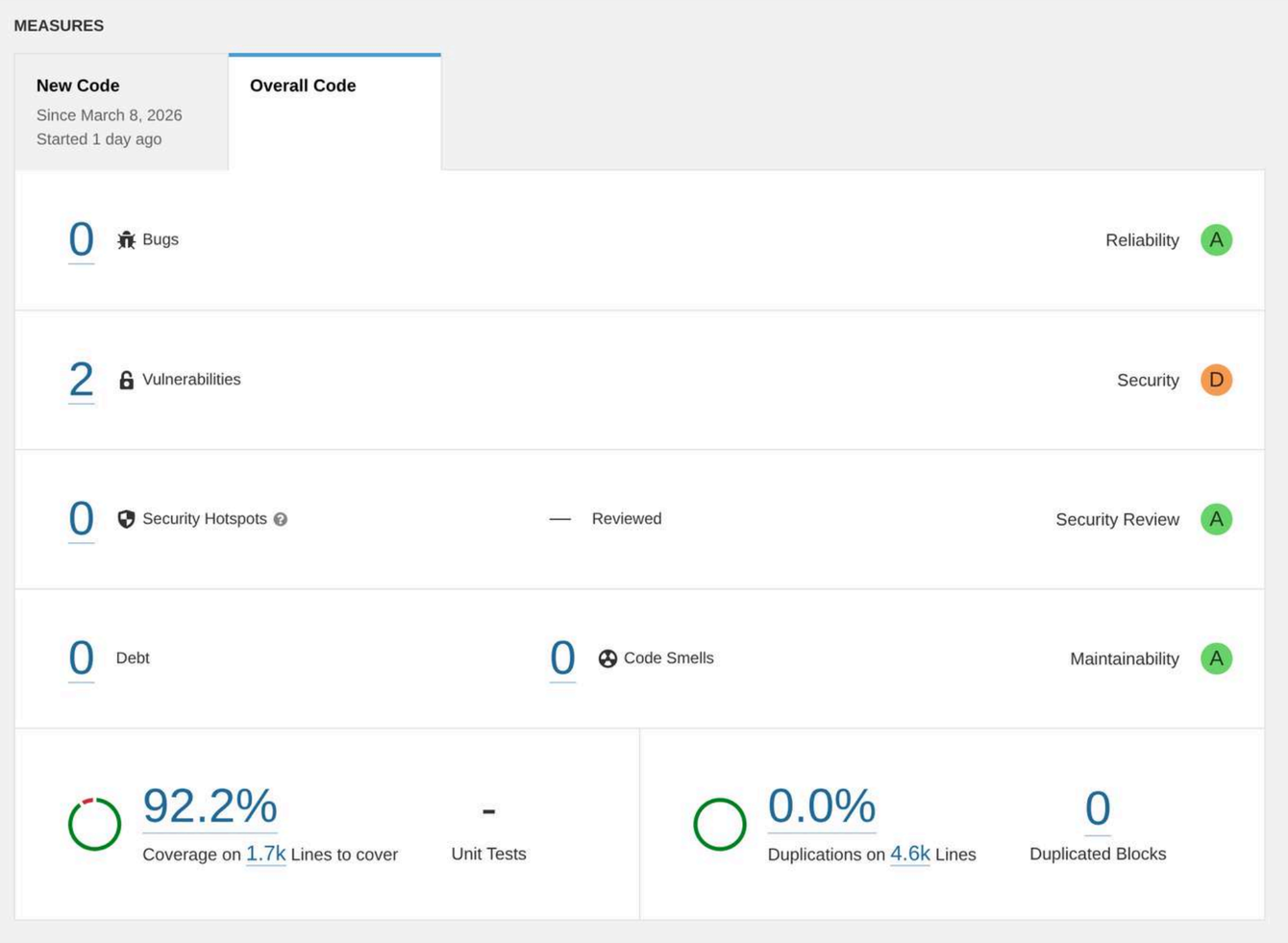
Pipeline

vuln-app-wazuh

Stage View

	Declarative: Checkout SCM	CI: Backend Tests & Coverage	CI: Frontend Tests & Build	SAST: SonarQube Code Analysis	GATE: SonarQube Quality Gate	DAST: OWASP ZAP Dynamic Scan	Declarative: Post Actions
Average stage times: (full run time: ~2min 26s)	1s	34s	19s	26s	171ms	1min 2s	388ms
#71 mar 09 01:39 1 commit	1s	34s	19s	26s	171ms	1min 2s	388ms

Pipeline - SAST



Pipeline - SAST

	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
Vuln App Wazuh							
frontend	3,967	0	0	0	0	92.1%	0.0%
vuln-api	625	0	2	0	0	93.0%	0.0%

2 Vulnerabilities

vuln-api/app/wazuh_client.py

Enable server certificate validation on this SSL/TLS connection. 2 months ago L14
Vulnerability Critical Open Not assigned 5min effort Comment cwe, owasp-a3, owasp-a6, owasp-m3, …

Enable server certificate validation on this SSL/TLS connection. 2 days ago L27
Vulnerability Critical Open Not assigned 5min effort Comment cwe, owasp-a3, owasp-a6, owasp-m3, …

2 of 2 shown

Pipeline - Quality Gate

Conditions

Conditions on New Code

Metric	Operator	Value
Coverage	is less than	80.0%
Duplicated Lines (%)	is greater than	3.0%
Maintainability Rating	is worse than	A (Technical debt ratio is less than 5.0%)
Reliability Rating	is worse than	A (No bugs)
Security Hotspots Reviewed	is less than	100%
Security Rating	is worse than	A (No vulnerabilities)

Pipeline - DAST

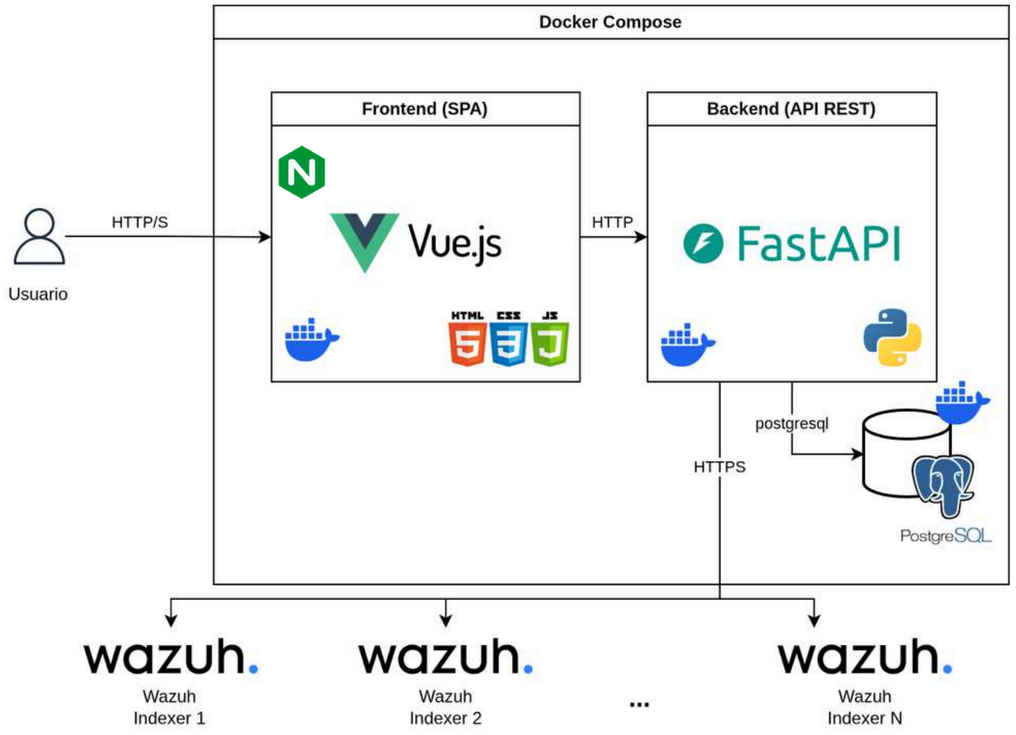
```
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 1
        http://api:8000/openapi.json (200 OK)
WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 1
        http://api:8000/openapi.json (200 OK)
FAIL-NEW: 0     FAIL-INPROG: 0  WARN-NEW: 2     WARN-INPROG: 0  INFO: 0 IGNORE: 0       PASS: 117
```

```
WARN-NEW: Cross-Origin-Embedder-Policy Header Missing or Invalid [90004] x 3
        http://frontend:80 (200 OK)
        http://frontend:80 (200 OK)
        http://frontend:80 (200 OK)
FAIL-NEW: 0     FAIL-INPROG: 0  WARN-NEW: 8     WARN-INPROG: 0  INFO: 0 IGNORE: 0       PASS: 59
```

Aspectos de Ciberseguridad

Seguridad en la arquitectura y el diseño

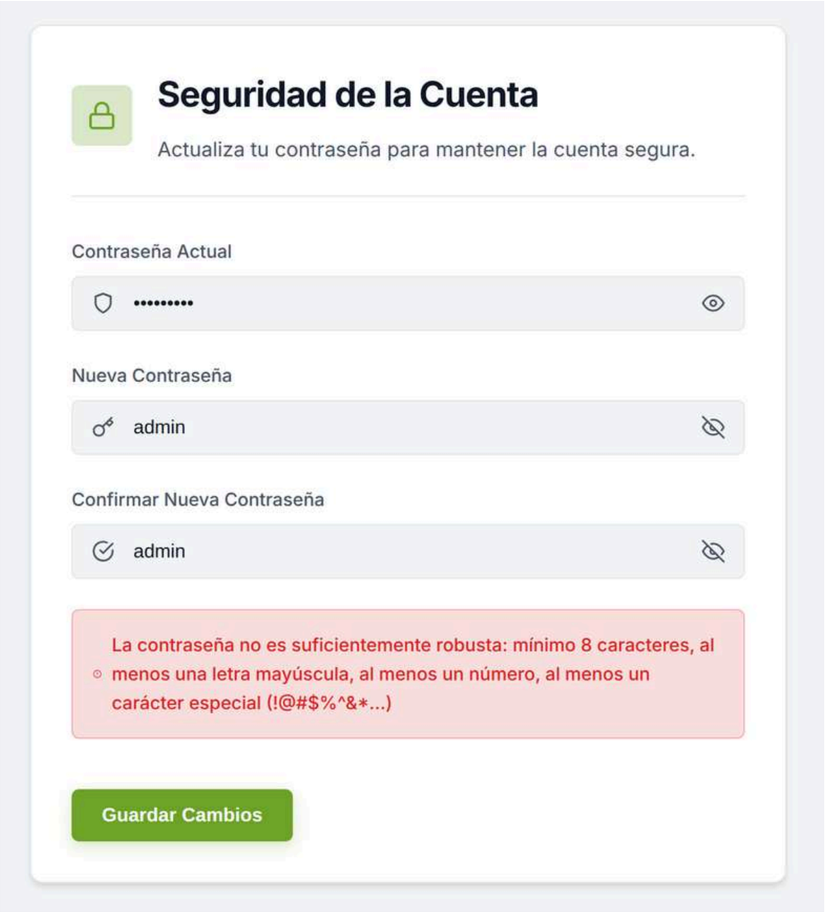
- Separación de capas: separación clara entre frontend, backend, base de datos, almacenamiento y servicios externos
- Principio de mínimo privilegio: cada componente solo tiene acceso a lo que necesita



Aspectos de Ciberseguridad

Control de acceso y autenticación

- Autenticación robusta:Contraseñas fuertes con políticas de caducidad y complejidad
- Gestión de sesiones: uso de tokens seguros (JWT con expiración) y revocación activa



Aspectos de Ciberseguridad

Control de acceso y autenticación

```python
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

- Autenticación robusta:Contraseñas fuertes con políticas de caducidad y complejidad
- Gestión de sesiones: uso de tokens seguros (JWT con expiración) y revocación activa

Aspectos de Ciberseguridad

Protección de datos sensibles

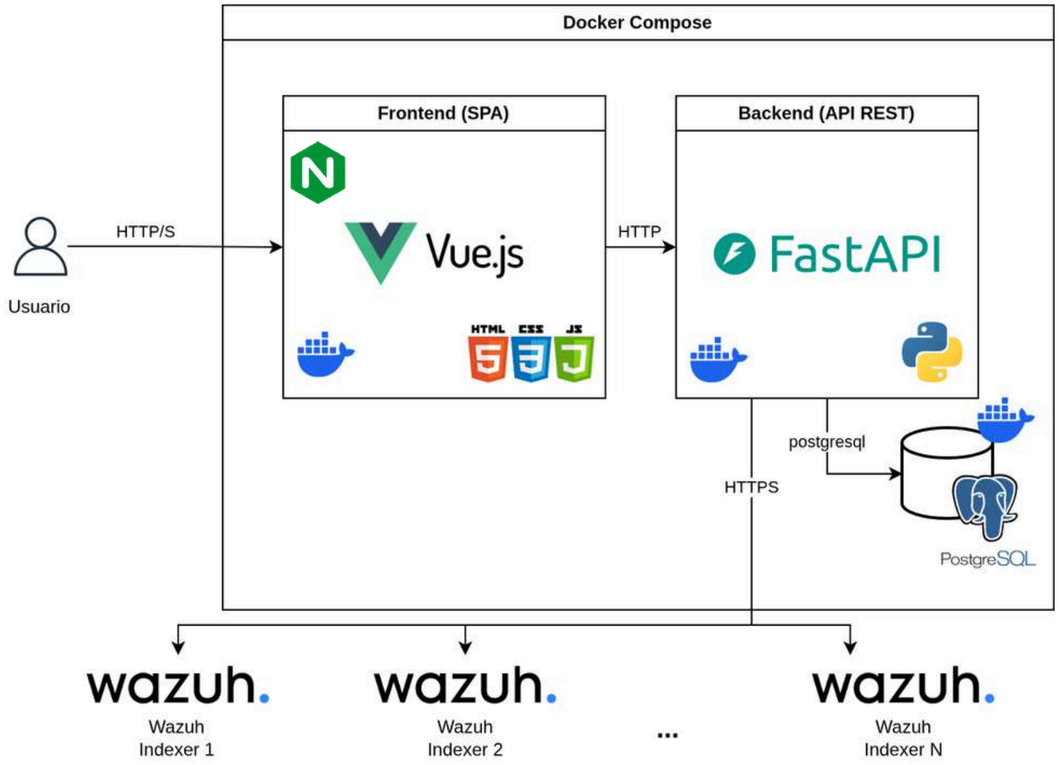
- Cifrado de datos en tránsito: uso obligatorio de HTTPS (TLS 1.2 o superior).
- Cifrado de datos en reposo: especialmente para PII, registros financieros o médicos

```python
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")


def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

def hash_password(password: str) -> str:
    return pwd_context.hash(password)
```

Aspectos de Ciberseguridad

Seguridad de la base de datos

- Acceso restringido solo desde el backend.
- Firewall de base de datos y controles de IP/puerto estrictos



Aspectos de Ciberseguridad

Pruebas y aseguramiento durante el ciclo de desarrollo

- Análisis estático (SAST) del código fuente.Análisis dinámico (DAST) en entornos de prueba o staging.
- Hardening de contenedores o entornos de ejecución (Docker, Kubernetes, etc.).

SonarQube

OWASP
Open Web Application Security Project

Despliegue

Guía de Despliegue en Producción

Este documento detalla los pasos necesarios para desplegar la aplicación en un entorno de producción, ya sea en un servidor con dominio público o en una red local utilizando certificados SSL autofirmados.

Requisitos Previos

Antes de comenzar, asegúrate de tener instalado lo siguiente en tu servidor:

- **Docker**
- **Docker Compose**
- **Puertos 80 y 443** abiertos y disponibles en el firewall de tu servidor.
- Si vas a usar un dominio real, asegúrate de que el registro DNS (A o CNAME) apunte a la IP de tu servidor.

Configuración Inicial

El proyecto utiliza un archivo `.env` en la raíz para gestionar las variables de entorno de los contenedores. Antes de ejecutar el proyecto, revisa y modifica las credenciales por defecto por motivos de seguridad:

```
VITE_API_URL=/api
POSTGRES_DB=vulnerabilidades_db
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
ENCRYPTION_KEY=_PDHhMGJ7ej8t4jZVgwX4xht5A8RBGwN4mwmUD15NkM=
SECRET_KEY=supersecret
```

Despliegue

Ejecución del Proyecto

El proyecto incluye un script interactivo llamado `iniciar_app.sh` que automatiza la configuración de red, la gestión de certificados SSL y el levantamiento de los contenedores Docker.

Para iniciar el despliegue, otorga permisos de ejecución al script y córrelo con permisos de administrador:

```
chmod +x iniciar_app.sh
sudo ./iniciar_app.sh
```

El script te presentará un menú con dos opciones:

Opción 1: Sin Dominio (Certificados Autofirmados / Local)

Ideal para entornos de desarrollo cerrado o redes locales.

- **Qué hace:** Detiene contenedores previos, copia la configuración de `docker-compose.nodomain.yml` y genera certificados SSL autofirmados usando `openssl`.
- **Cronjobs:** Si existía una tarea de renovación automática de certificados Let's Encrypt de despliegues anteriores, el script la elimina de forma segura.
- **Servicios:** Levanta la base de datos (PostgreSQL 15), la API y el Frontend, exponiendo los puertos 80 y 443.

Opción 2: Con Dominio (Certificados Let's Encrypt / Prod)

Esta es la opción recomendada para producción real expuesta a internet.

- **Qué necesitas:** El script te pedirá ingresar tu dominio (ej. `midominio.cl`) y un correo electrónico para los avisos de expiración de Let's Encrypt.
- **Qué hace:** 1. Copia la configuración desde `docker-compose.domain.yml` y `nginx.domain.conf`.

2. Solicita un certificado SSL oficial utilizando un contenedor temporal de Certbot en modo "standalone".
3. Levanta la infraestructura (`db-api` , `api` , `frontend` y un contenedor permanente de `certbot`).

- **Renovación Automática:** Si la obtención del certificado es exitosa, el script instala automáticamente un Cronjob en el sistema anfitrión (`# VULN_APP_CERTBOT_RENEWAL`) que ejecutará diariamente a las 03:00 AM el comando de renovación de Certbot y recargará el servidor Nginx para aplicar los cambios.

Despliegue

Arquitectura de Servicios

En entorno de producción con dominio, Docker Compose levantará los siguientes servicios interconectados a través de la red `app-network`:

1. **db-api:** Base de datos PostgreSQL 15 con un volumen persistente `postgres_api_data`.
2. **api:** Backend construido desde la carpeta `./vuln-api`, con un *healthcheck* configurado que espera a que la base de datos esté lista antes de iniciar.
3. **frontend:** Servidor Nginx que expone los puertos 80 y 443, construido desde `./frontend`. Tiene montados los volúmenes para leer los certificados de Let's Encrypt.
4. **certbot** (Solo Opción 2): Contenedor encargado de gestionar las renovaciones futuras de SSL.

Preguntas